

Plugin Architectures with Rust

an Analysis of Obstacles
and Prospects

20.05.2026



THU
Technische
Hochschule
Ulm

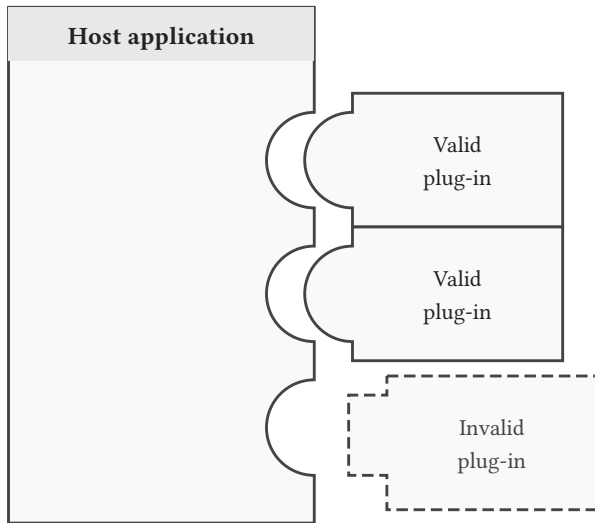
1	Introduction	3
1.a	Plugin Systems	4
1.b	Criteria	5
2	ABI	6
2.a	Application Binary Interface	7
2.b	Dynamic linking and loading	8
2.b.a	Process of dynamic linking and loading	8
2.b.b	Example with Rust ABI	9
2.c	IPC	11
3	Conclusion	13
3.a	Performance	14

1 Introduction

Plugin Systems



- **Extensibility:** The ability to add new functionality post-deployment.
- **Modularity:** Plugins are isolated units that interact with the host through a well-defined interface.
- **Dynamic Loading:** Plugins are typically loaded at runtime rather than linked statically at compile time.



Criteria



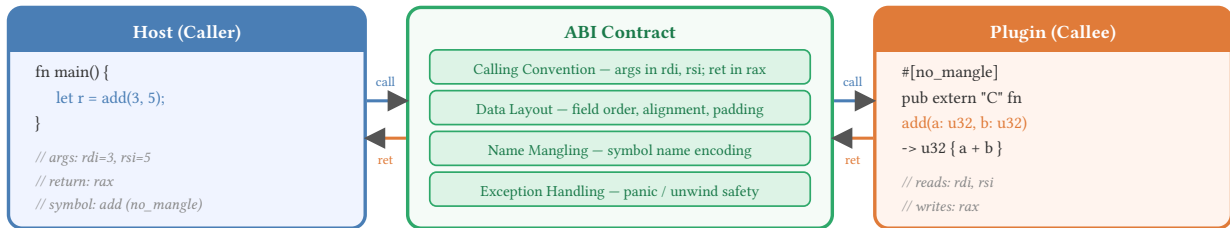
- Performance Overhead
- Development Complexity
- Language Limitations
- Interoperability

2 ABI

Application Binary Interface



What an ABI Governs



Data Layout: struct { u8, u32, u8 }

■ field byte ■ padding

#[repr(C)] — 12 bytes



#[repr(Rust)] — may be 8 bytes



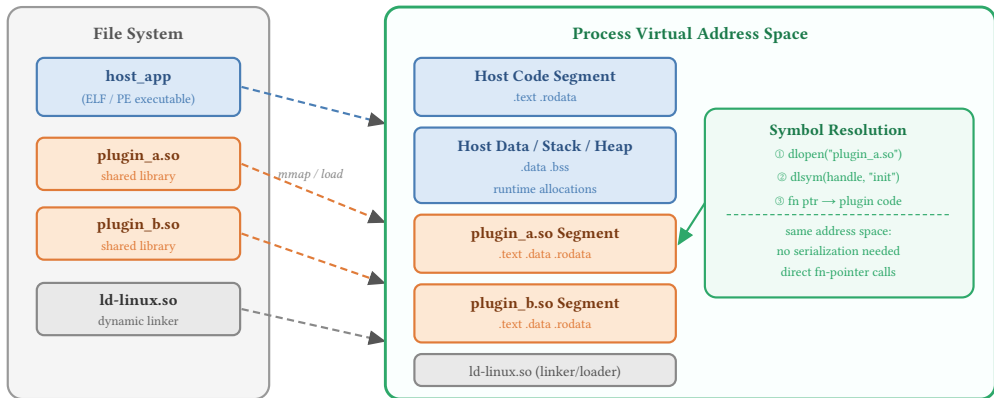
⊠ The compiler may reorder fields — a C caller reading offset 8 would get garbage

Dynamic linking and loading



2.b.a Process of dynamic linking and loading

- 1) Discovery
- 2) Loading
- 3) Resolution
- 4) Execution



Dynamic linking and loading (ii)



2.b.b Example with Rust ABI

Using the `libloading` crate:

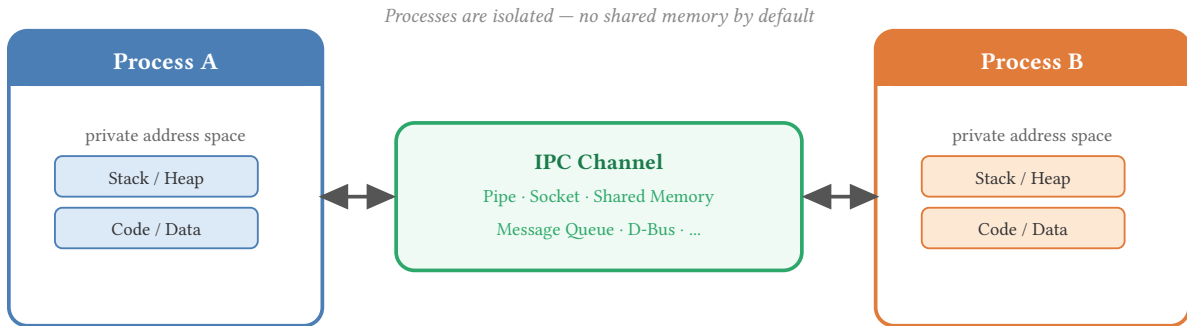
```
trait Fuser {
    fn fuse(&self, data: &[SensorData]) -> Result<SensorData, Box<dyn Error>>;
}
struct Plugin<T> where T: Fn() -> Box<dyn Fuser> + Copy{
    lib: Library,
    func: T,
}
fn new(path: &PathBuf, symbol_name: &[u8]) -> Result<Self, libloading::Error> {
    let lib = unsafe { Library::new(path) }?; // 1+2) Discovery and Loading
    let symbol: Symbol<T> = unsafe { lib.get(symbol_name) }?; // 3) Resolution
    let func: T = *symbol; // fn deref(&self) -> &T
    Ok(Plugin { lib, func })
}
```

Dynamic linking and loading (iii)



```
type FuseFunc = extern "Rust" fn() -> Box<dyn Fuser>;
let p: Plugin<FuseFunc> = Plugin::new(&plugin_path.into(),
"average_plugin".as_bytes()).unwrap();
a(&p);

fn a(plugin: &Plugin<FuseFunc>) {
    let fuser: Box<dyn Fuser> = plugin.get();
    let sd = [ SensorData::new(42.0, 0.01), ... ];
    let res = fuser.fuse(&sd); // 4) Execution
    println!("a {:?}", res);
}
```



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat.

Term
Definition

3 Conclusion

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	9%	0	0	+
rust_bridge (JSON)	537%	+	0	++
rust_bridge (Binary)	25%	-	0	++

Table 1: Summary of results.